

齊魯工業大學

本科毕业论文（设计）

基于 Next.js 的全栈网上书店管理系统的设计与实现

学部（学院） 计算机科学与技术学院

专业班级 计科 22-2

学生姓名 郝鸣

学号 202203010050

导师姓名 吕国华 周策

2026 年 5 月 28 日

本科毕业论文（设计）

基于 Next.js 的全栈网上书店管理系统的设计与实现

学部（学院） 计算机科学与技术学院

专业班级 计科 22-2

学生姓名 郝鸣

学号 202203010050

导师姓名 吕国华 周策

2026 年 5 月 28 日

齐鲁工业大学本科毕业论文（设计）

原创性声明

本人郑重声明：所呈交的毕业论文（设计），是本人在指导教师的指导下独立研究、撰写的成果。论文（设计）中引用他人的文献、数据、图件、资料，均已在论文（设计）中加以说明，除此之外，本论文（设计）不含任何其他个人或集体已经发表或撰写的成果作品。对本文研究做出重要贡献的个人和集体，均已在文中作了明确说明并表示了谢意。本声明的法律结果由本人承担。

毕业论文（设计）作者签名： 希鸣

2026 年 05 月 28 日

齐鲁工业大学本科毕业论文（设计）

使用授权说明

本毕业论文（设计）作者完全了解学校有关保留、使用毕业论文（设计）的规定，即：学校有权保留、送交论文（设计）的复印件，允许论文（设计）被查阅和借阅，学校可以公布论文（设计）的全部或部分内容，可以采用影印、扫描等复制手段保存本论文（设计）。

指导教师签名： 吕国华 周策 毕业论文（设计）作者签名： 希鸣

2026 年 05 月 28 日

2026 年 05 月 28 日

目 录

摘 要	I
ABSTRACT	II
第 1 章 绪论	1
1.1 选题背景和意义	1
1.2 国内外研究现状	1
1.2.1 国外研究现状	1
1.2.2 国内研究现状	2
1.2.3 国内外研究现状总结	3
1.3 研究内容	3
1.3.1 研究内容	3
1.3.2 研究目标	4
1.4 论文组织架构	4
第 2 章 系统需求分析	6
2.1 系统可行性分析	6
2.1.1 技术可行性分析	6
2.1.2 经济可行性分析	6
2.1.3 操作可行性分析	7
2.2 系统功能需求分析	7
2.2.1 用户端需求分析	7
2.2.2 管理员端需求分析	8
2.3 系统业务流程分析	9
2.3.1 用户购书业务流程	10
2.3.2 管理员业务流程	11

2.4 系统非功能需求分析	12
第3章 系统设计	13
3.1 系统总体架构设计	13
3.1.1 系统架构模式	13
3.1.2 系统技术架构	13
3.1.3 系统整体架构设计	14
3.2 系统功能模块设计	15
3.2.1 用户认证模块设计	15
3.2.2 图书展示模块设计	16
3.2.3 搜索与分页模块设计	17
3.2.4 购物车模块设计	18
3.2.5 订单模块设计	19
3.2.6 用户中心模块设计	20
3.2.7 后台管理模块设计	21
3.3 数据库设计	23
3.3.1 数据库概念结构设计	24
3.3.2 数据库逻辑结构设计	24
3.4 系统安全设计	27
3.4.1 JWT 身份认证机制	27
3.4.2 密码加密存储机制	27
3.4.3 权限控制机制	28
3.4.4 XSS 风险防护机制	28
3.4.5 安全请求头与 CORS 控制	28
第4章 系统实现与测试	30
4.1 开发环境搭建	30
4.2 系统测试	30

4.2.1 功能测试	30
4.2.2 接口测试	31
4.2.3 安全测试	32
4.2.4 测试结果分析	33
第 5 章 总结与展望	34
参考文献	36
致 谢	37
附 录	39

摘 要

随着电子商务平台以及移动互联网的发展，网络上的在线购书已成为人们获取书籍的主要方式之一。传统的书店管理系统一般会面临前后端分离的成本高、各个功能之间相互关联紧密导致开发难度大、后期维护困难等问题，但是使用当下流行的全栈框架搭建出一家网上书店可以很好地解决这些问题，在一个技术栈的基础上既可以让用户有更好的购书体验又能提高书店管理人员的工作效率。因此本文主要针对“基于 Next.js 的全栈网上书店管理系统”的设计与开发进行研究并实现了一家包含用户端及管理端操作流程的书店网站。

本项目基于 Next.js 14 进行开发，使用 React 18 进行页面组件化，通过 App Router 和 Route Handlers 对页面及接口进行集中式管理；在数据存储上采用了 Redux Toolkit 存储用户信息及购物车数据，使用 MySQL 数据库保存系统相关数据并通过 Prisma 对其进行读写等操作，保证数据的一致性；同时加入了 JWT 身份认证、内容安全策略以及输入内容过滤等措施，在一定程度上增强了系统的安全性及健壮性。

根据网上书店的需求，系统具有用户注册、登录、图书分类浏览、关键词搜索、图书详情展示、购物车管理、在线购买、查看订单历史以及管理收货地址的功能，同时管理员可进行图书信息更新、库存及价格管理、订单处理、用户信息查看以及销售数据统计等工作。

通过上述实验可以看出，在 Next.js 上进行全栈开发可以有效提升开发效率，同时有利于保证系统的统一性以及后续维护工作，对于学习和使用电子商务类 Web 系统具有一定的借鉴作用。

关键词：Next.js；网上书店；全栈开发；MySQL；Prisma；电子商务系统

ABSTRACT

The rapid development of e-commerce and web application technologies has made online bookstores one of the critical channels in which readers can search, buy and manage books. In contrast to conventional monolithic or poorly coupled bookstore systems, a full-stack architecture using recent React frameworks will enhance the efficiency of the development process, consistency of interaction, as well as maintainability. Thus, the paper develops and executes a full-stack online bookstore management system on Next.js to fulfill the roles of the customer-oriented functions and the administrator-oriented management functions.

The system is based on Next.js 14 and React 18. App Router and Route Handlers are used to structure pages and RESTful APIs into the single framework. Global state management is performed using Redux Toolkit, particularly user state and shopping cart state. MySQL is chosen as the central relational database and Prisma ORM is used to offer type-safe data access and schema management. In order to increase the security of the system, this project incorporates JWT-based authentication, content security policy, and input sanitization. The implemented business functionalities are user registration and login, book classification and search, book details display, related recommendation, shopping cart administration, order submission, order history search, profile and address administration, and administrator side book administration, stock adjustment, order processing, user administration and sales statistics.

The current paper also involves the requirement analysis, architecture design, database design, key module implementation, and system testing. The project has been tested and validated in real-life MySQL setup which proves that the created system is capable of supporting the main workflow of an online bookstore with transparent structure, timely interaction and high scalability. The research demonstrates that full-stack development on the basis of Next.js is appropriate during practical training and creation of web applications with an emphasis on engineering.

Key words: Next.js; online bookstore; full-stack development; MySQL; Prisma; e-commerce system

第 1 章 绪论

1.1 选题背景和意义

在图书零售业务方面，由于线下书店受到自身库存量、门店面积以及营业时间限制，在一定程度上不能满足读者对于快速查找及即时购买需求，而网上书店将图书展示、搜索以及交易移至互联网环境中，则是从“线下分散交易”转变为“线上集中处理”，但是一些具体的技术问题仍然需要解决。

目前一些现有的 Web 购物平台更多使用传统的前后端分离或者单页应用的方式，在首页的加载速度以及被搜索引擎抓取的能力上不尽如人意；另外前后端各自独立开发和部署也会增加对接口进行管理和合作的成本，造成整个项目的开发周期长并且运行效率低的问题。所以需要引入一种既可以实现服务端渲染又能有良好的项目结构来解决这些问题。

Next.js 是一个基于 React 全栈框架，利用服务器端渲染(SSR)、静态站点生成(SSG)和服务端组件等方式把一部分页面渲染放到服务端完成，减轻前端渲染压力同时加快首屏加载速度，在工程上统一路由以及数据抓取方式使得前后端代码可以放在同一个项目里面编写，避免由于拆分而导致的复杂性问题，降低开发及维护的成本^[1]。

根据以上思路，笔者开发一款基于 Next.js 网上书店网站，在此平台上可以查看书籍、搜索书籍、添加到购物车以及下单等功能。该系统使用前后端一体化的方式进行开发，既保证系统的功能丰富性又提升了开发效率并且有利于后续对页面进行优化以及实现基本的 SEO 功能。

这说明，在电商类 Web 系统中利用服务端渲染技术和统一全栈框架相结合的方法可以解决传统 Web 应用所面临性能以及开发复杂性之间的冲突，可作为中小型电商系统的一种实施方式。

1.2 国内外研究现状

1.2.1 国外研究现状

在线图书销售系统中，主要问题在于系统性能、开发难度以及可扩展性之间的平衡，在前端，传统的单页应用方式由于首屏加载使用的是前端渲染的方式，在一定程度上会造成首屏延迟的问题，而且对于搜索引擎来说不利于被索引；而在后端，前后端分离开发一般需要分别管理接口和服务端页面，这无疑加大了系统的耦合性和协作的工作量。

为了解决这些问题，一些电商平台开始采用基于分布式架构和服务化的解决方案，

并借助推荐技术和分析工具对用户的浏览以及购买行为进行建模，提高商品的相关性和系统的性能，在这种平台中，图书搜索及推荐一般由用户行为所决定的排序实现，在一定程度上提高了查找的速度^[2]。

从前端技术角度来看，像 React、Vue 这类基于组件化的框架把页面划分为可以复用的小部分来减少复杂的交互所要付出的努力。而 React 由于它的虚拟 DOM 以及它拥有的庞大的生态系统所以在中大型网站上被广泛使用并且是前端工程化的一个重要基础。^[18]

在此之上，Next.js 是一个基于 React 全栈框架，在此基础上引入服务器端渲染（SSR）及静态站点生成（SSG），将一部分页面生成工作放到服务端进行，降低首屏时浏览器端处理负担。此外，统一的路由以及数据获取方式使得前后端可以在一个项目里实现，避免由于接口拆分过多造成的问题以及前后端数据不同步问题。

从应用角度，以上技术组合已应用于电商平台、内容管理系统等多种类型的 Web 应用开发工作中，在不影响业务的基础上，通过对渲染以及工程组织进行优化提升页面加载速度并且减少系统的开发及维护成本。

1.2.2 国内研究现状

国内在线图书销售系统一般以图书搜索、分类浏览、购物车管理、订单流转以及支付接口为主要内容，形成一个完整购物流程。而这些网站日常要处理大量客户请求并且经常需要进行页面跳转，这对其响应速度以及后期开发成本都是一个不小的挑战。

早期网上书店一般采用 JSP、SSM 这类传统的 Java Web 开发方法进行开发，使用服务端渲染结合模板化页面生成网页。虽然这种方法有较好的工程稳定性，但是由于前端与后端耦合度较高，所以一旦需要对页面做小改动或是增加新功能就需要对后端代码进行相应调整，工作量较大。而随着电商行业的发展壮大，这种模式在版本更新时所遇到的问题越来越突出^[3]

。为了解决以上问题，后续的系统工程逐渐发展成为前后端分离模式，在前端方面开发者更愿意使用 Vue 或者 React 来负责前端交互，而后端则是以 RESTful API 的形式提供数据支持。这在一定程度上解决了团队协作的问题，但是随之而来的问题是接口的设计、跨端调试、多个部署等问题^[4]。

而在这种情况下，Next.js 作为 React 生态系统中全栈解决方案提供了一个整合方案。由于它自带的服务器端渲染(SSR)以及静态站点生成(SSG)，可以使得大量页面的内容在服务端就已经完成，从而减少客户端渲染的工作量^[5]。此外，它不按常理出牌的一

体化路由以及数据获取方式，也就自然而然地简化了跨项目间的状态同步以及接口对齐的问题^[6]。

1.2.3 国内外研究现状总结

纵观目前研究以及工程化发展过程，国外相关平台倾向于较早采用分布式、微服务化的设计理念，在模块高度解耦以及基于数据驱动的推荐等方面已经形成较为完善的技术体系，同时有关 React 及其全家桶 Next.js 的应用也十分广泛。而我国的网上书店类网站开发工作仍然主要集中在传统的 Java Web 框架或经典的前后端分离模式上^[7]。

而在传统的前后端分离或者传统的 Java Web 方案中，在前端和后端之间进行分离或者过度耦合都会带来一系列问题：如接口联调的成本较高；页面生成需要消耗大量的浏览器端计算资源；整个流程较长等都会影响到系统的版本更新速度以及首屏加载时间。

为了解决以上问题，本文采用 Next.js 全栈框架实现网上书店项目。这并不是说要重写整个项目的业务流程，而是利用 Next.js 将前端页面交互以及后端深层次的功能合并到一个项目中进行统一管理，并且使用 SSR 和 SSG 提供一部分渲染的工作量到客户端。这样一方面减少代码重复以及协调工作量，另一方面可以降低首屏白屏时间以及提高搜索引擎抓取友好性^[8]。

通过对当前研究成果进行分析可以看出，在国内有关 Next.js 等新型全栈框架应用于图书电商方面的研究较少。因此，为了适应现今 Web 工程架构的发展趋势即向集约化、全栈化发展，开发基于 Next.js 的网上书店系统具有重要工程实践意义及现实借鉴意义。

1.3 研究内容

1.3.1 研究内容

本文以“基于 Next.js 全栈网上书店管理系统”为主要研究对象，在分析目前电商发展趋势及网上书店系统中相关业务的基础上，对系统功能、数据库设计以及关键技术进行阐述并完成整个项目的设计与开发过程。

在系统需求分析过程中，我们首先对网上书店具体业务流程进行研究，在了解一般用户及管理员使用需求基础上确定系统功能架构及其业务流程。系统主要包括用户端和管理员端两种类型，用户端主要是实现图书浏览、检索、购物车管理和下单等操作；管理员端主要用于图书信息管理、订单管理和用户信息管理等。

在系统设计上，根据当前流行的 Web 应用开发方式^[9]，在项目中使用 Next.js 全栈

框架进行开发，使用 React 实现前端页面组件化开发，同时使用 Next.js Route Handlers 来处理后端接口问题。对于数据存储问题，则使用 MySQL 数据库来存储系统中的相关业务数据，并借助 Prisma ORM 来进行数据库的操作以及对数据模型进行定义。

在系统实现过程中，实现用户注册登录、图书分类展示、关键词搜索、购物车管理、在线下单、订单查询以及后台图书管理等功能并使用 JWT 进行身份认证及权限控制来保证系统的安全。

在系统测试中，对用户登录、图书检索、购物车操作、订单生成以及后台管理等功能进行测试，从而保证整个系统完整、稳定及可用。

1.3.2 研究目标

为了满足网上书店系统实际业务需求，提高系统开发效率与用户使用体验，本文主要围绕以下几个方面开展研究与实现。

- （1）实现完整的用户购书流程。
- （2）实现后台管理功能。
- （3）提高系统开发效率。
- （4）保证系统安全性。

1.4 论文组织架构

本文以“基于 Next.js 全栈网上书店管理系统”为题进行研究与设计，在系统需求分析、架构设计、功能实现及系统测试等方面进行阐述，对整个系统的研发工作进行了探讨和总结。全文主要包括五部分内容。

第 1 章：绪论。

主要介绍课题研究背景与意义、国内外研究现状、研究内容与研究目标、研究方法以及论文整体组织结构，为后续系统研究与实现奠定理论基础。

第 2 章：系统需求分析。

主要对网上书店系统的可行性进行分析，并结合用户与管理员两类角色，对系统功能需求、业务流程以及非功能需求进行研究，为系统设计提供依据。

第 3 章：系统设计。

主要介绍系统总体架构设计、功能模块设计、数据库设计以及系统安全设计，对系统实现过程中涉及的关键技术与模块进行分析与说明。

第 4 章：系统实现与测试。

主要是系统开发环境及其实现的过程，包括用户端及管理员端的功能实现并通过功能测试、性能测试及安全性测试来检验系统的运行情况。

第 5 章：总结与展望。

对本文的研究内容以及系统的实现成果进行总结，在此基础上对目前系统存在的问题进行剖析并对以后系统的发展方向及趋势提出建议。

第 2 章 系统需求分析

2.1 系统可行性分析

2.1.1 技术可行性分析

系统的技术可行性分析主要针对目前所使用的开发技术能否达到系统的要求并且能便于以后的开发及后期的维护工作而进行的研究，在了解本项目的实际情况的基础上，本项目采用 Next.js 14、React 18、Redux Toolkit、Prisma ORM、MySQL 等技术搭建整个框架，基本可以完成网上书店系统的设计与开发工作。

对于前端开发，本项目使用 Next.js 14。相比于传统的前端框架，Next.js 不仅支持页面路由，还可以进行服务端渲染以及 API 开发，可做到前后端一体。在前端界面部分使用 React 18 实现^[10]。React 是一种基于组件的方式进行开发的语言，可以把整个页面拆分成若干个模块来单独管理，有利于提高工作效率并且方便后期添加新功能。为了保证跨页面的状态一致性，本项目使用 Redux Toolkit 对整个项目的状态进行统一管理^[12]。而在数据获取上，我们采用了 Prisma ORM 来操作数据库。相比于手写 SQL，使用 Prisma ORM 可以利用模型映射的方式来完成数据读取和表之间关系的操作，简化开发工作量。数据库使用 MySQL 存储数据。由于 MySQL 是比较成熟的数据库管理系统，在数据安全性和查询速度等方面都表现良好，可以用于存放用户信息、书籍信息、订单信息及购物车信息等主要业务的数据。

以上所述，本系统所采用的技术较为成熟，相关技术文档齐全，也符合系统对页面交互、数据存储、状态管理和权限管理等要求，所以具有良好的技术可行性。

2.1.2 经济可行性分析

经济可行性分析主要是对系统的建设和运行所花费的成本是否合理进行评价，同时考察该项目有无较高的投资回报率。

本系统使用的 Next.js 14、React 18、Redux Toolkit、Prisma ORM 和 MySQL 全部为免费开源技术框架或者工具，不需要支付额外的版权费用，在开发成本上，系统使用前后端一体的方式进行开发，开发人员可以利用 Next.js 完成前端页面以及后端接口的开发工作，在一定程度上降低了由于不同的技术栈之间转换所带来的学习成本和维护成本；在部署及运行的成本上，因为本系统是基于 B/S 架构的应用程序，所以只需要用浏览器就可以访问，不需要下载安装客户端等。

从总体上分析，本系统的开发费用以及维护费用都比较低，所采用的技术方案具有

良好的经济性，满足中小型网上书店系统的开发要求，所以有很好的经济性。

2.1.3 操作可行性分析

操作可行性分析是考察系统的易用性，即该系统是否方便用户使用，用户是否能很快熟悉系统的操作等。

对于界面的设计上，本系统采用了比较简洁的设计风格，在用户的界面上主要是以图书的查看、检索、购买等功能为主导而展开的设计工作，使得整个页面更为清晰明了；为便于系统的兼容性问题，项目中使用了响应式的设计方式，即页面会根据不同的屏幕大小来自适应变化其显示的内容，在台式机、平板电脑和平板设备上的浏览器中都能获得良好的体验；另外本系统是基于 B/S 架构开发而成，在客户端不需要安装任何的应用程序，只需要使用浏览器就可以访问到该系统。

综上所述，该系统的界面友好、操作简便易懂并且支持多种设备使用，所以有很好的实用性。

2.2 系统功能需求分析

系统功能需求分析主要用于明确系统应实现的业务功能，并为后续系统设计与开发提供依据。

2.2.1 用户端需求分析

用户端是网上书店系统的主体部分，主要是让用户能够实现从浏览书籍到购买的一系列操作并且有良好的用户体验以及信息管理功能。

（1）注册与登录需求

为使系统能辨别出用户身份并且有权限管理，系统应具有注册以及登录的功能，在用户第一次进入系统的时候可以选择通过邮箱或用户名和密码来进行注册，系统会对用户的这些信息进行检查是否合理，然后把密码加密之后存放到数据库中。

（2）图书浏览需求

图书浏览是用户使用系统最基本操作，在系统中展示所有已上传图书同时可以按图书类别筛选以便用户找到自己感兴趣图书。

（3）图书搜索需求

为使用户更快获取所需书籍信息，在系统中需有关键词查询的功能，用户可依据书名、作者或者一些内容等关键词进行查找，系统根据查找到的信息向其展示相关的书籍内容，节省用户翻阅时间。

（4）购物车需求

购物车功能主要是用于帮助用户暂存要购买的商品，是商品浏览到下单之间的一个环节，在用户浏览书籍时可以添加自己想要购买的书本至购物车，并且还可以对购物车内的商品进行增加、删除、修改或查询等操作。

（5）订单需求

订单模块主要用来让用户进行在线购买的操作，在用户选择好收货地址之后可以点击在线购买并根据购物车的商品信息创建相应的订单。

（6）订单查询需求

为便于用户查询以往购买情况，系统需有订单查询功能，在个人中心可查看本人以往所有订单信息，如订单状态、商品明细、订单总价及收货地址等。

（7）个人中心需求

个人中心主要是供用户进行个人信息及账号相关操作，应该能够使用户可以修改头像、用户名等。

2.2.2 管理员端需求分析

管理员端主要是为系统管理员服务，主要目的是进行图书数据维护、订单处理、用户管理和运营管理等操作以便使网上书店系统正常运转并能满足日常工作需要。

（1）管理员登录需求

为保证后台数据安全，系统须设有管理员登录功能，对后台访问权限进行控制，在管理员登录时，系统除了要进行一般用户的身份验证之外，还需要检查用户的角色是否为管理员，只有管理员才能进入到后台管理页面。

当一般用户试图访问后台资源时，系统应当禁止其访问或者跳转到后台登录页面，以免非法用户进入管理后台而造成后台信息泄露等安全问题。

（2）图书管理需求

图书管理功能是后台系统重要组成部分，主要是为了方便在系统内添加、修改、删除或者上架下架书籍而设计，以便及时更新书籍信息。

对于增加新书，在添加书籍时，管理员可以填写书名、作者、价格、库存、分类及描述等；而在编辑时，也可以根据需要对书籍信息进行修改。如果书籍暂时不出售或者不再出售，则系统也应该有相应的功能来使书籍上下架操作，以便管理员更好地管理商品信息。

（3）库存与价格管理需求

库存以及价格是图书销售中重要信息，所以系统应有相关功能使管理员可以随时更新商品销售情况。

管理员可以依据实际情况修改图书库存以及价格等信息，在库存改变后，系统要保证库存与订单的一致性，防止由于库存问题造成超卖现象发生，而价格管理可使管理员随时根据市场变化灵活定价，使系统更具活力。

（4）订单管理需求

订单管理是用于查看系统中所有订单的状态并且可以监控订单处理的过程，管理员可查看所有客户提交的订单并可以根据订单的不同情况进行相应的操作如未支付、已支付、已发货、完成或者取消等。

利用订单状态进行维护，可以及时了解订单流动情况，进而提升工作效率，同时也让客户知道订单进展情况，给客户良好体验。

（5）用户管理需求

为了让管理员了解系统中用户状况，系统需有用户管理功能。管理员可以查询系统内所有注册用户的资料，例如用户名、邮箱地址、注册时间和订单数等信息以便进行后台管理工作。

此外，用户管理模块还要使管理员能够了解用户的使用状况，以便日后的工作开展以及对系统的运营进行分析。

（6）销售统计需求

销售统计是本系统后台管理的一个重要部分，也是系统实现数据化运营的一个很重要的功能。系统要对用户的数量、图书的数量、订单的数量以及销售额等重要信息进行统计并汇总在后台首页上让管理员可以一目了然地看到整个网站的情况。

2.3 系统业务流程分析

图 2-1 展示了网上图书商城系统的顶层功能拓扑。系统在垂直方向上划分为用户端与管理端两大核心控制域：

用户端以交易闭环为中心，横向衍生出账号认证、图书浏览、图书详情、购物车管理、订单支付、订单查询以及个人中心 7 大核心功能板块。

管理员端以运营支撑为中心，横向涵盖账号认证、图书管理、库存管理、订单管理、用户管理以及数据统计展示 6 大后台管控模块。该架构图为后续系统详细设计与业务流程的具体实现提供了全局性的结构指导。

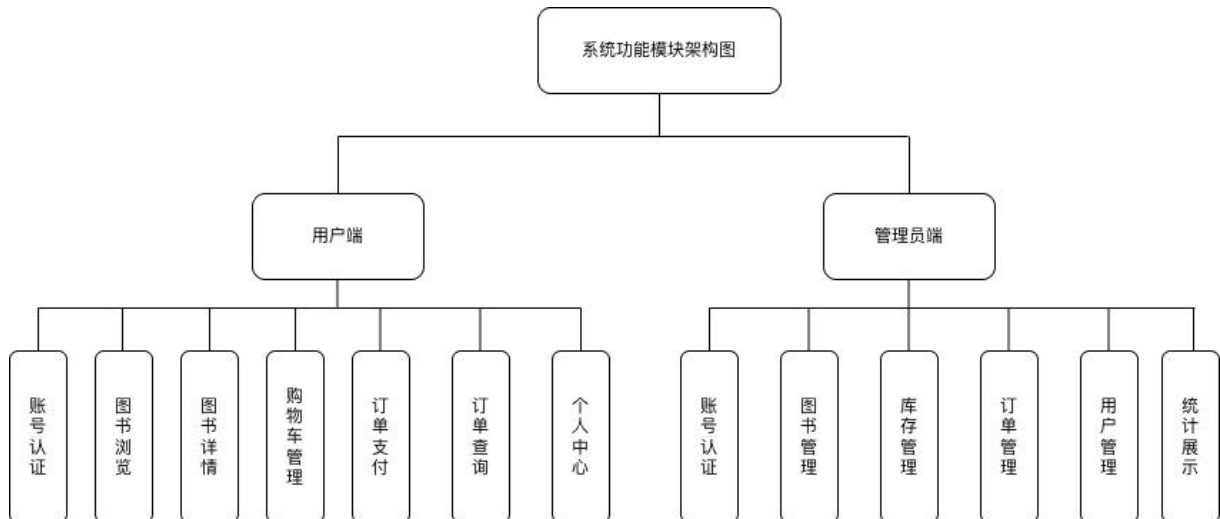


图 2-1 系统功能架构图

2.3.1 用户购书业务流程

如图 2-2 所示，用户购书流程是系统的核心业务流程，主要包括用户注册登录、图书浏览、加入购物车、提交订单及订单管理等步骤，具体流程如下：

用户首先进入系统首页，在未登录状态下可浏览部分图书信息。当用户需要进行购买操作时，需先完成注册与登录流程。登录成功后，系统基于 JWT 对用户身份进行认证，并为用户分配访问权限。

随后用户可以通过首页推荐或搜索功能查找目标图书。在选定图书后，用户可将图书加入购物车，并在购物车页面对商品数量进行调整或删除操作。购物车数据在前端进行状态管理，并与后端保持同步。

当用户确认购买后，进入订单提交流程。系统会根据购物车内容生成订单数据，并写入订单表（orders）与订单详情表（order_items）中，同时生成唯一订单编号。用户完成“模拟支付”操作后，订单状态更新为“已支付”。

最终，用户可以在“我的订单”页面查看订单列表及详细信息，包括订单状态、商品信息及支付状态等，实现完整的购书业务闭环。^[2]

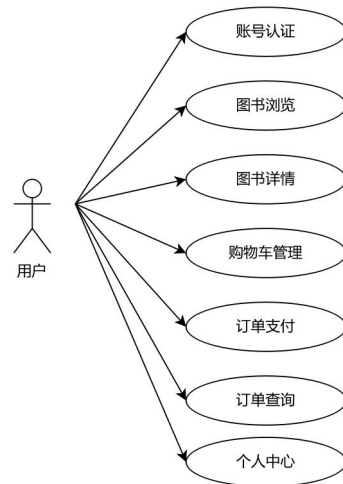


图 2-2 普通用户用例图

2.3.2 管理员业务流程

如图 2-3 所示，“管理员”作为后台管控的主参与者，业务流程主要围绕系统内容管理与订单管理展开，是保障系统正常运行的重要组成部分。

管理员首先通过后台登录入口进行身份验证，系统通过 JWT 对管理员身份进行校验，确认权限后进入管理后台界面。

在图书管理模块中，管理员可以对图书信息进行新增、修改、删除及上下架操作，相关数据实时同步至数据库中的 **books** 表。同时，管理员可以对图书分类进行管理，以保证前台展示内容的结构化与规范性。

在订单管理模块中，管理员可以查看所有用户订单信息，包括订单编号、用户信息、商品明细及订单状态。管理员可根据业务需求对订单进行状态更新，例如标记为已发货或已完成。

此外，管理员还可以对用户信息进行基础管理，例如查看用户列表及处理异常账户等操作，从而保障系统整体运行安全与稳定。

通过以上流程，管理员实现了对系统内容与业务数据的统一管理，为整个网上书店系统的正常运行提供了支撑。

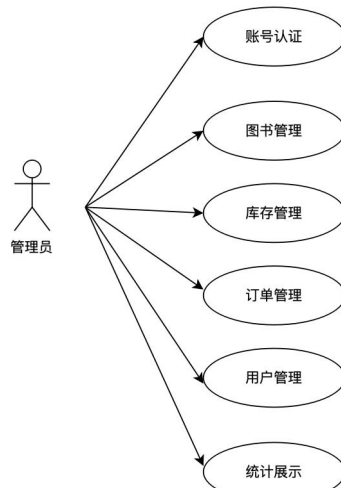


图 2-3 管理员用例图

2.4 系统非功能需求分析

因为网上书店系统包含用户的账号、订单信息和个人收货地址等隐私内容，在开发过程中要特别注重安全问题并使用一些方法减少一般 Web 安全威胁。

在身份认证上，使用 JWT 进行身份认证来维护用户会话。当用户登录时，服务器向客户端发送一个身份令牌，在之后的所有请求中都需要这个令牌来进行身份验证从而防止未经授权的访问。

对于权限控制，在权限上要区别对待普通用户以及管理员的角色。普通用户只能看到自己相关的业务信息，而管理员具有后台管理系统功能的能力，这样就可以做到不同的人有不同的权限，防止越权现象的发生。

为保障用户账号安全，系统对用户的密码采用加密方式进行保存，在用户注册时使用 bcrypt 对密码进行一次哈希操作，防止密码以明文形式存在，如果数据库被泄露也会减少用户的个人信息被泄露的可能性。

对于常见的脚本注入攻击，系统需有 XSS 防御能力，在前端方面利用 React 默认转义来避免恶意脚本运行，在后端方面对接收用户提交的数据进行简单的清理工作也能有效防止非法脚本被存入数据库中。

另外，系统还采用了 Security Headers（安全请求头）提高浏览器安全性，比如禁止页面嵌套、禁止资源类型嗅探以及控制资源加载来源等以降低被恶意攻击的风险。

对于跨域访问控制问题，在此过程中系统要使用 CORS 来限制允许访问来源、请求方式及请求头等信息，以使接口调用是安全可控的，防止恶意跨域请求给系统造成危害。

第 3 章 系统设计

3.1 系统总体架构设计

3.1.1 系统架构模式

本系统基于 B/S（Browser/Server，浏览器/服务器）模式设计与实现。用户端以及管理员端都使用浏览器访问该系统，不需要下载客户端软件，在 Next.js 环境下同时部署前端页面、后端逻辑以及接口等。

在这种结构中，客户端主要是显示页面并且与用户进行交互，而服务端则是处理业务逻辑、授权和保存数据等任务。相比于传统的 C/S（Client/Server）架构，B/S 架构部署和管理更加简便，在系统需要更新时不需要经常给客户端做更新，而且也便于以后添加新功能。而对于网上书店系统来说，采用这种方式可以支持多种设备访问并且有利于以后对系统进行维护以及更新版本。

3.1.2 系统技术架构

本项目是基于 Next.js 全栈开发方式，使用 React、MySQL、Prisma 实现整个前后端一体的应用程序来完成用户登录注册、图书管理、购物车、订单等功能。

对于前端开发，在前端开发中，本项目采用 Next.js 14 和 React 18 开发具有组件化的用户界面，利用 React 组件复用来提升页面开发效率以及代码可维护性。应用 App Router 路由方式，以 app 文件夹为核心进行组织，根据不同情况进行前后端分离开发，使得页面结构和功能分离。

从后端的角度看，应用使用 Next.js 提供的 Route Handlers 来创建 RESTful API，在 app/api 下完成用户登录、图书搜索、购物车操作、订单处理以及后台管理等功能；同时用 Redux Toolkit 管理用户的登录状态以及购物车中的商品信息，以便在不同页面之间保持一致性及即时性^[11]。

在数据持久化上，系统使用 MySQL 作为关系型数据库来保存用户的账号、图书的信息、订单信息以及收货地址等相关重要信息，在提高开发数据库的速度的同时也保证了对数据的安全性，系统使用 Prisma ORM 将数据库中的表映射成实体对象，方便进行复杂的查询、事务的处理以及表之间的关系等操作。

从安全认证来看，系统使用 JWT（JSON Web Token）进行身份验证。当用户登录时，服务器生成一个 token 并保存到 HttpOnly Cookie 中，从而保证无状态会话，增加安全性并且减少服务器 Session 的压力。

3.1.3 系统整体架构设计

从整个系统的实现上来说，本项目包括了表示层、业务逻辑层、数据访问层和安全防范层四个层次，在这四个层次之间互相配合共同实现整个系统的功能。

表示层主要是由 Next.js 页面、React 组件^[12]以及 Redux 状态管理组成，用于展示页面以及与用户进行交互。如图书展示、分类筛选、加入购物车、提交订单及管理员相关页面等，都在此部分实现。利用组件化的方式来组织页面内容可以提高代码复用性，也可以方便后期开发^[13]。

业务逻辑层主要是 Route Handlers 接口以及业务相关的一些辅助函数，负责整个系统的业务逻辑，如用户的登录认证、订单生成、商品库存更新以及权限管理等工作，所有的前端请求都经过这个层进行处理后返回给前端，从而保证系统的正常运行^[14]。

数据访问层使用 Prisma Client 对接 MySQL 数据库进行增删查改，在订单创建等重要环节利用数据库事务来保证一系列操作的一致性，防止出现商品库存扣减失败或者订单信息错误等情况发生。

安全保障层主要是通过系统中间件及安全工具函数实现，如 JWT 校验、CSRF 防护、输入参数校验、安全请求头设置和跨域相关等，以提升系统的安全性。

系统的请求流程为：当浏览器发起页面请求或者接口请求时，中间件先处理请求并设置一些安全相关的参数，然后生成一个相应的 CSRF Token；之后 API 接口会对用户的登录状态以及这个请求是否合法进行校验，然后 Prisma 去数据库里把需要的数据获取出来，最后统一格式后返回给前端页面进行展示以及状态的变化等。

系统整体架构如图 3-1 系统整体架构图所示。

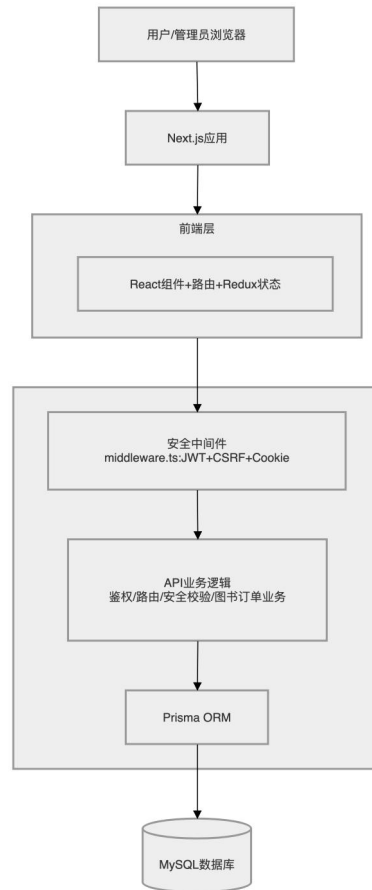


图 3-1 系统整体架构图

3.2 系统功能模块设计

3.2.1 用户认证模块设计

用户认证模块是系统进行用户身份鉴别及访问控制的一个重要环节，在用户注册、登录以及登出等方面起到一定作用的同时也起到对系统进行权限检查的基础作用。

在用户注册时，系统会对接收到的邮箱、用户名以及密码等基本信息进行合法性校验，防止非法数据写入到数据库中，在对密码进行处理时，使用 **bcrypt** 算法对密码进行加盐哈希后再保存，以防止密码明文存储，如果数据库被窃取，则可以减少由于用户的账户信息泄露所造成的损失。如图 3-2 所示。

用户登录时，系统先判断邮箱以及密码是否正确，验证成功之后从服务器返回一个有效期为 7 天 JWT Token 并用 HttpOnly Cookie 存储该 Token。因为 HttpOnly Cookie 不能被浏览器 JavaScript 获取，所以可以在一定程度上避免 XSS 攻击导致 Token 泄露从而保证系统的会话安全性。如图 3-3 所示。

因为后台管理涉及到较高权限的操作，所以系统还增加了一个管理员的身份校验，在有管理员的角色才能看到后台页面以及相关接口资源，一般用户不能进入后台，防止

非法人员进入系统对系统的数据产生不良的影响。

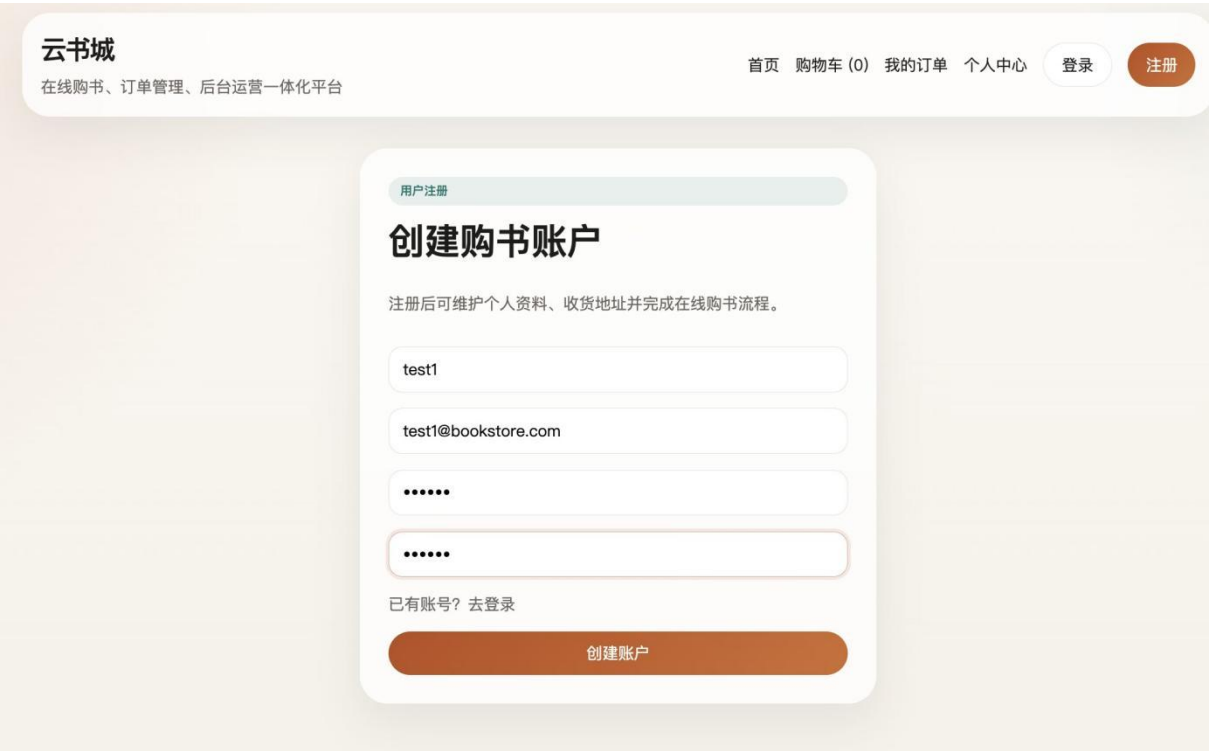


图 3-2 注册页面

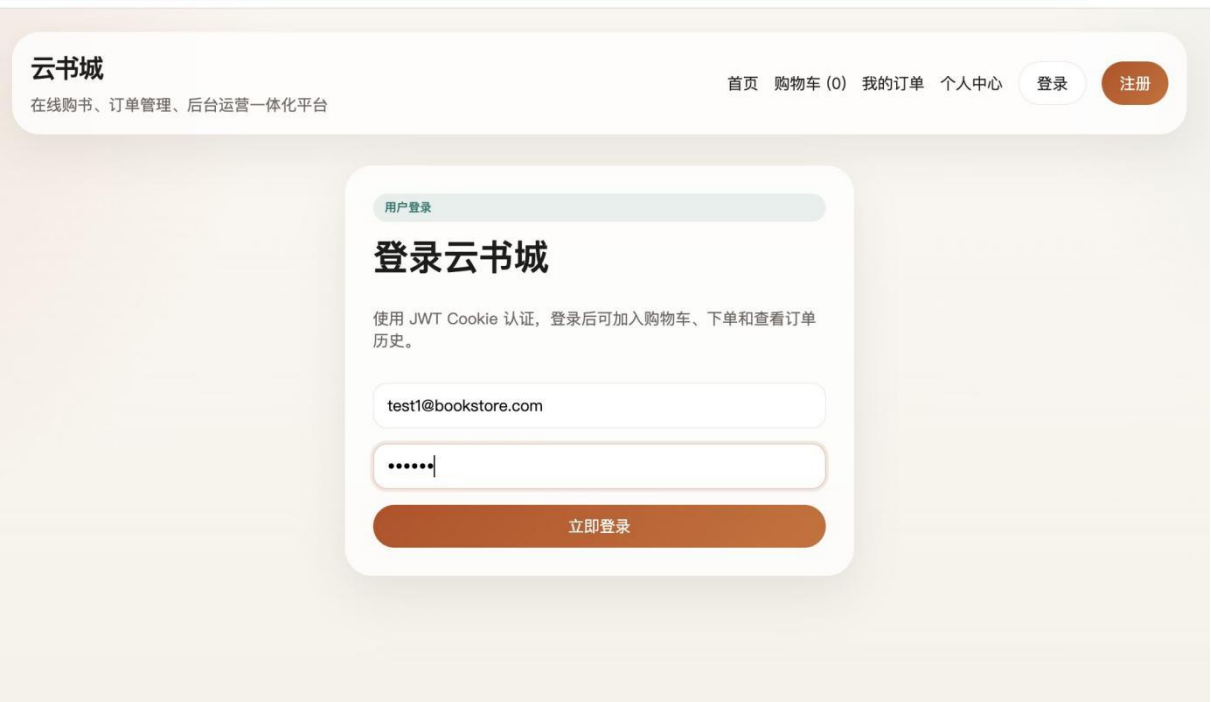


图 3-3 登录页面

3.2.2 图书展示模块设计

图书展示模块是网上书店系统的重要部分，在整个网站中起着至关重要的作用，负责书籍信息展示及商品浏览等工作，如首页展示、分类查询、图书详细信息浏览等以及

后台图书管理等，如图 3- 4 所示。

在前台首页上，系统会把所有状态为已发布（PUBLISHED）的书单显示出来并且可以按书单类型来筛选，便于读者找到想要的书单。与只有一种列表形式相比，分类查询可以降低读者找书难度，在一定程度上提高阅读体验。

图书详情页展示一本图书所有信息，例如封面、作者、售价、库存量等，同时也会根据该书所在类目为其推荐相似书籍，增加页面内容的相关性，提升用户的购买意愿。

后台部分主要是针对管理员开放，在此可以添加、编辑、删除图书并可以上架、下架等等，还可以维护价格、库存等相关信息以便商品信息能够及时更新。

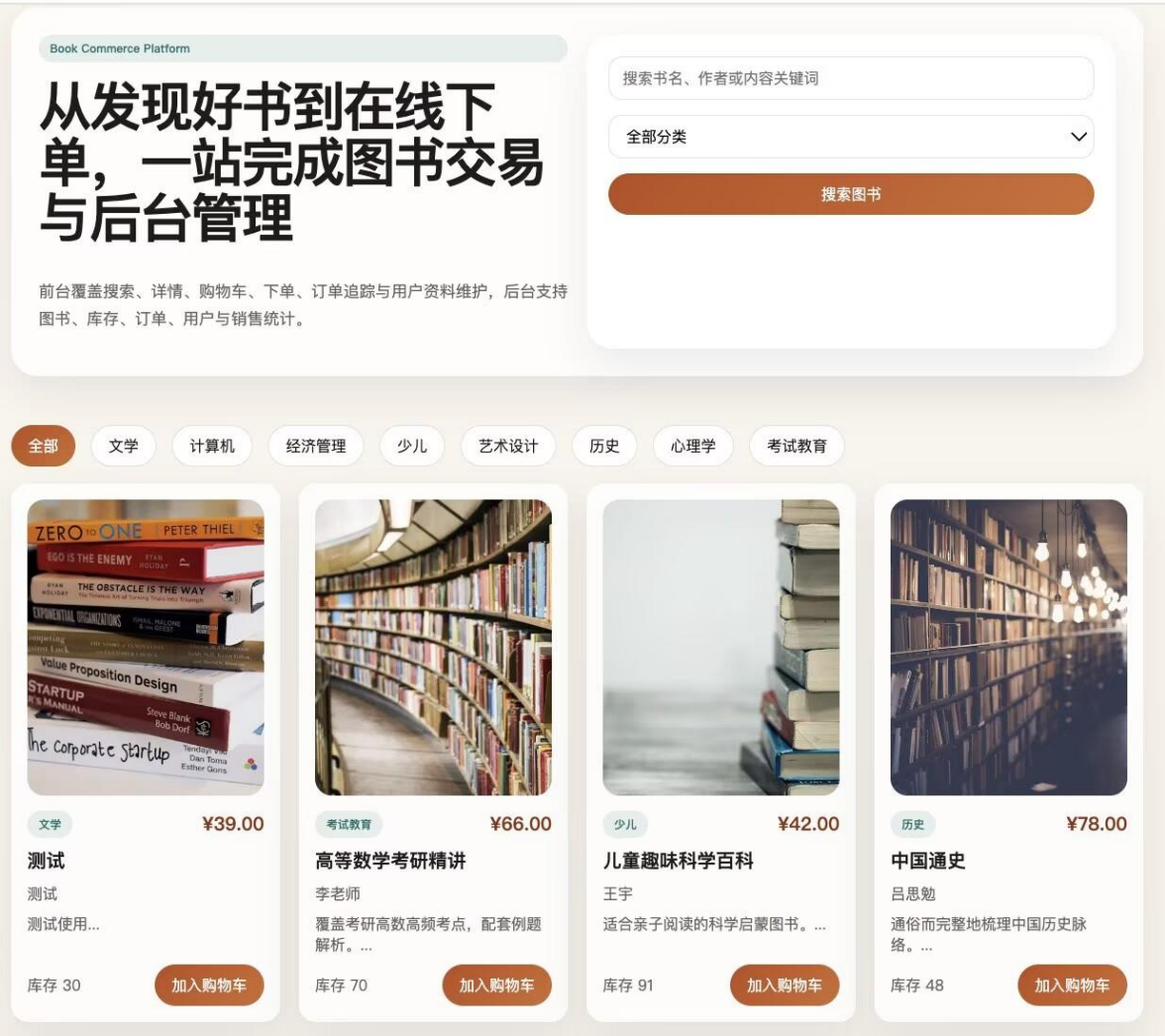


图 3- 4 图书展示页面

3.2.3 搜索与分页模块设计

为了提高图书检索速度，系统实现关键词搜索以及分页，便于读者更快定位所需资料。

在搜索方面，系统可以针对书名、书作者或者书简介等进行模糊搜索，在用户输入

相关内容之后，系统会根据搜索内容给用户相应书籍，节省用户一页页翻找时间。如图 3-5 示。

对于分页问题，系统使用 `page` 和 `pageSize` 进行分页，一般一页显示 8 条数据并且统一返回分页格式便于前端进行页码以及翻页操作。分页可以减少一次请求的数据量也可防止页面加载大量信息导致加载慢等问题。



图 3-5 搜索图书页面

3.2.4 购物车模块设计

购物车模块用于存放用户所选购商品并为接下来的结账做准备，具有添加到购物车、修改数量、删除商品等功能。

用户登录后，可以收藏自己喜爱书目至购物车中，在此过程中为了避免一本图书被多次加入购物车的情况发生，我们为用户 ID 以及图书 ID 设置联合主键来保证一个用户

不能有两份相同的购物车信息并且在数量上进行累计。

当用户改变购买数量后，系统也会立即检查现有的库存情况，保证商品的数量不会大于库存的最大值从而避免出现超卖的情况发生。而且系统还会不断计算出购物车内商品的数量并在页面上方进行更新，让用户可以随时了解到自己所处的状态，提高用户体验。如图 3-6 所示。



图 3-6 购物车页面

3.2.5 订单模块设计

订单模块是整个系统业务流程的关键所在，在系统中起到的作用就是生成订单、对订单进行管理和查看等操作。

在用户提交订单前，系统会对购物车商品信息、收货地址及商品库存情况进行校验，保证提交订单信息正确无误。为防止订单创建时发生错误造成数据问题，系统使用数据库事务，在一个事务内完成订单插入、订单明细生成、库存减少、销量增加以及购物车清空等工作，以保证业务一致性^[1]。如图 3-7，图 3-8 所示。

目前系统使用模拟支付方式进行交易，在用户下单之后，系统会自动将该笔订单的状态变为已付款。一般用户可以在个人中心中看到自己所有的订单信息，但是作为管理员具有更大的权利，可以查看所有订单并可对订单进行操作，如发货、完成或者取消等。

订单结算

支持默认地址选择、订单备注与模拟支付。

收货地址

test1 13310689503
山东省济南市历城区齐鲁工业大学

支付方式

模拟支付

订单备注

确认订单

中国通史 x 1 ¥78.00

总计 **¥78.00**

提交订单并支付

图 3-7 订单结算页面

历史订单

可查询订单状态、支付信息与下单明细。

BS1780558819224739
2026/06/04 15:40
中国通史 x 1
支付方式: 模拟支付

已支付
¥78.00
¥78.00

图 3-8 支付成功页面

3.2.6 用户中心模块设计

用户中心模块主要是用于用户的个人资料管理及收货地址管理，给用户更全面的账号服务。

对于个人信息，在个人中心中可对用户名、头像等进行更改以满足个人需要。在收货地址中可添加、修改或删除地址信息并且还可以设置一个默认地址来避免每次购买商品时重复输入信息而加快下单速度。如图 3-9 所示。

为了防止用户修改地址而造成的历史订单信息丢失，在生成订单的同时将当时收货人信息作为快照保存在该订单内。之后如果地址改变，但是之前产生的订单仍然可以查看到最初收货人信息，保证订单的信息真实性和准确性。

个人中心

维护个人资料、头像与收货地址。

用户资料

可更新昵称与头像地址，头像上传接口可对接文件上传组件。

test1@bookstore.com

test1

头像 URL（或调用 /api/upload/avatar 上传）

保存资料

收货地址维护

支持新增、设为默认与删除地址。

test1 13310689503

默认地址

删除

山东省济南市历城区齐鲁工业大学

收件人

手机号

省份

城市

区县

邮编

详细地址

☐ 设为默认地址

新增地址

图 3-9 个人中心页面

3.2.7 后台管理模块设计

后台管理模块主要是供管理员使用的，进行系统的日常运维及数据管理和业务操作等。

系统在进入后台页面前需对管理员进行验证，如果当前用户不是管理员则直接跳转到后台登陆页面以免非管理员用户查看不该看到的内容。如图 3- 10 所示。

管理员进入后台之后，就可以进行图书管理、订单管理、用户管理和数据分析等工作，在图书管理中主要是对商品信息进行编辑，如图 3- 11 所示。在订单管理中是对订单的状态进行处理，如图 3- 12 所示。在用户管理中查看用户的详细资料以及账号信息，如图 3- 13 所示。

另外，系统设有统计看板页面，可显示用户人数、图书总数、订单数、总销售额及最新一笔订单等信息，供管理员更清晰掌握系统状况并有助于后期工作开展。

21



图 3- 10 管理员后台页面

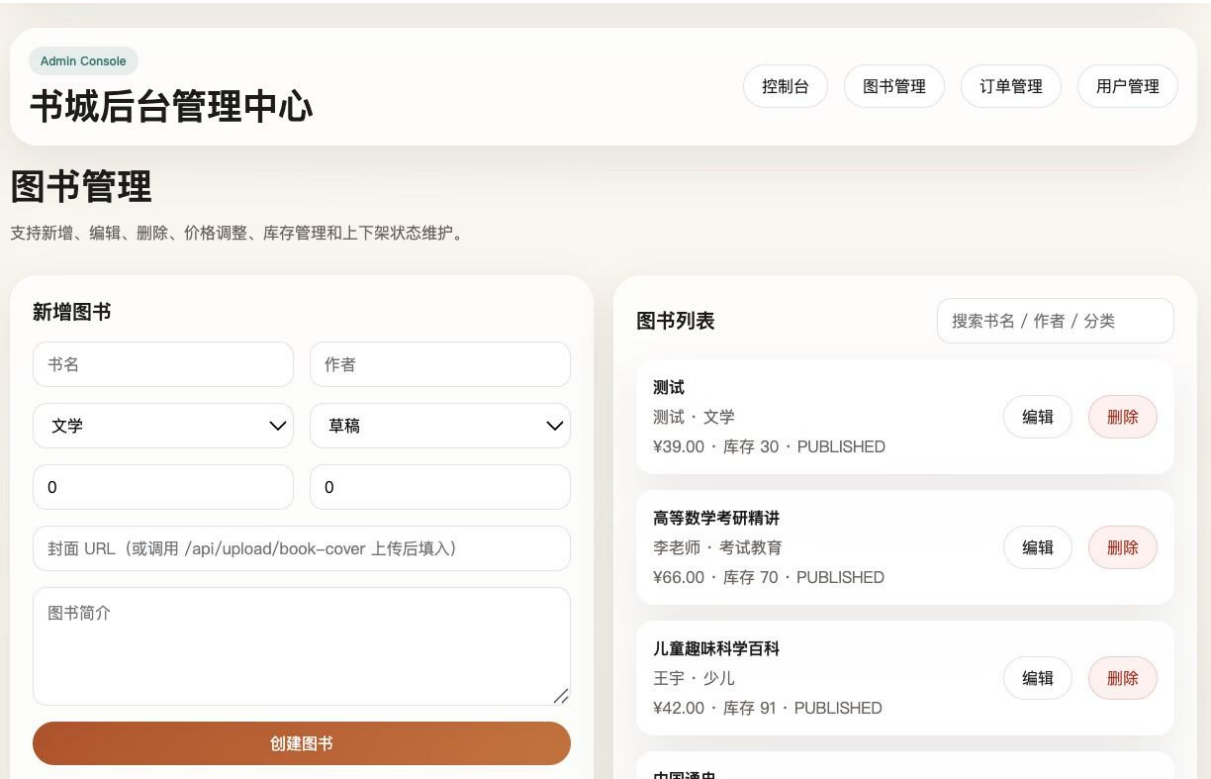


图 3- 11 图书管理页面



图 3- 12 订单管理页面

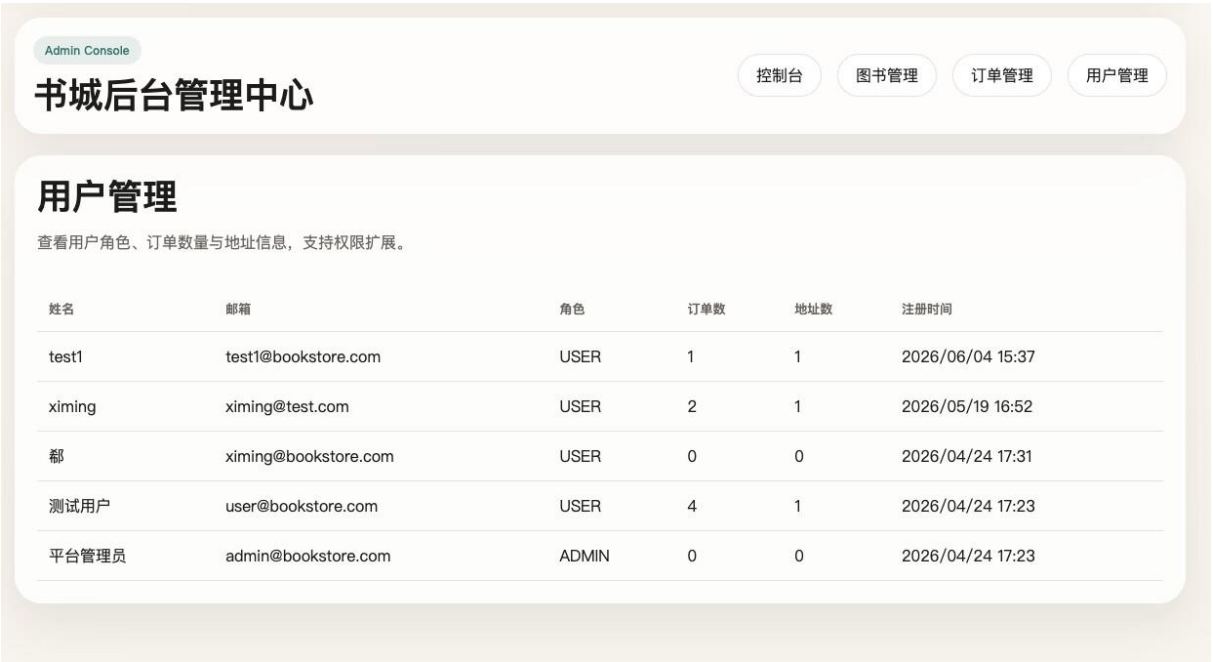


图 3- 13 用户管理页面

3.3 数据库设计

3.3.1 数据库概念结构设计

数据库概念结构设计主要是为了表示系统中的各个实体之间联系，在此基础上进行进一步逻辑设计及数据库建立工作。根据网上书店系统功能要求和数据间相互作用关系，本系统以用户购物为中心设计了一系列重要实体，如 `users`、`books`、`categories`、`cart`、`orders`、`order_items`、`address` 等。

其中，`users` 实体用于存储用户的个人信息并与其他系统内的多个实体有关联，在一个系统中，一个用户可以有若干个收货地址，所以 `users` 与 `address` 是一对多的关系；另外，在浏览商品时，用户可以加入多个商品到购物车，所以 `users` 与 `cart` 也是一对多的关系。而在订单上，一个用户可以生成多个历史订单，所以 `users` 与 `orders` 也是成一对多的关系。

图书信息主要是通过 `books` 表来管理。为便于对图书进行归类，系统在 `categories` 和 `books` 之间创建了一对多的关系，即一个分类可以有很多本书，但是每一本书只能有一个分类。因为用户可以把书放入购物车，所以 `books` 和 `cart` 也是一对多的关系，也就是说一本书可以在很多个用户的购物车内出现。

订单部分由 `orders` 以及 `order_items` 两个实体构成，`orders` 存储订单相关信息，而 `order_items` 存储订单内商品信息。一般情况下一个订单会有多个商品，所以 `orders` 和 `order_items` 之间是一对多的关系；另外一本书也有可能出现在不同的订单中，因此 `books` 和 `order_items` 之间也是一对多的关系。

3.3.2 数据库逻辑结构设计

在实现数据库的概念结构之后，需要把实体关系转化为具体的物理表结构，在这里使用 MySQL 作为关系型数据库并用 Prisma ORM 将数据模型进行统一映射来支持用户、图书、订单以及购物车等功能。

（1）用户表（`users`）

用于保存用户的个人信息以及认证信息，主键是 `id`。主要有 `name`、`email`、`password`、`role`、`avatar_url`、`created_at`、`updated_at` 等字段，其中 `email` 设置唯一性来防止重复注册，`password` 保存的是经过加密的密码，`role` 用来判断是不是普通用户或者管理员从而进行基本的权限管理。

表 3-1 用户表

字段名	中文名	类型	键/约束	说明
<code>id</code>	用户编号	String	主键	用户唯一标识

name	姓名	String	—	用户姓名
email	邮箱	String	唯一	用户登录邮箱
password	密码	String	—	密码哈希值
role	角色	UserRole	—	用户角色
avatarUrl	头像地址	String	—	用户头像 URL
createdAt	创建时间	DateTime	—	数据创建时间
updatedAt	更新时间	DateTime	—	数据更新时间

(2) 图书表（books）

用于维护图书基本信息，主键是 id，可以通过 category_id 和分类表进行关联。有 title、author、description、price、stock、sales、cover_url、status、published_at 等字段，在 slug 上设置唯一性以使每本书都有一个唯一的访问路径。

表 3-2 图书表

字段名	中文名	类型	键/约束	说明
id	图书编号	String	主键	图书唯一标识
title	书名	String	—	图书名称
slug	图书短标识	String	唯一	友好 URL 标识
author	作者	String	—	图书作者
description	简介	Text	—	图书描述信息
price	定价	Decimal(10,2)	—	销售单价
stock	库存	Int	—	库存数量
sales	销量	Int	—	累计销量
coverUrl	封面地址	String	—	图书封面 URL
category	分类	String	—	图书类别
status	状态	BookStatus	—	发布状态
publishedAt	发布时间	DateTime	—	上架发布时间
createdAt	创建时间	DateTime	—	数据创建时间
updatedAt	更新时间	DateTime	—	数据更新时间

(3) 收货地址表（address）

用于存储用户的收货地址信息，通过 user_id 和 users 表关联。包含字段有 receiver, phone, province, city, district, detail, postal_code, is_default, is_default 用于表示是否为默认地址。

表 3-3 收货地址表

字段名	中文名	类型	键/约束	说明
id	地址编号	String	主键	地址唯一标识
userId	用户编号	String	外键→User.id	所属用户
receiver	收件人	String	—	收货人姓名
phone	联系电话	String	—	联系方式
province	省份	String	—	省级行政区
city	城市	String	—	市级行政区
district	区县	String	—	区县级行政区
detail	详细地址	String	—	街道门牌等

postalCode	邮编	String	—	邮政编码
isDefault	是否默认	Boolean	—	默认地址标识
createdAt	创建时间	DateTime	—	数据创建时间
updatedAt	更新时间	DateTime	—	数据更新时间

（4）购物车表（cart）

用于存储用户的购物车中的商品信息, id 为主键, user_id 连接 users 表, book_id 连接 books 表, 有 quantity, created_at, updated_at 字段以及 (user_id, book_id) 联合唯一性约束防止重复添加相同的书。

表 3-4 购物车项表

字段名	中文名	类型	键/约束	说明
id	购物项编号	String	主键	购物项唯一标识
userId	用户编号	String	外键→User.id	关联用户
bookId	图书编号	String	外键→Book.id	关联图书
quantity	数量	Int	—	加入购物车数量
createdAt	创建时间	DateTime	—	数据创建时间
updatedAt	更新时间	DateTime	—	数据更新时间

（5）订单表（orders）

用于存放订单基本信息, 主键是 id, 由 user_id 连接到 users 表。有 order_no、total_amount、payment_method、payment_status、status、note、address_snapshot、created_at、updated_at 等字段。并且对 order_no 进行唯一性限制, address_snapshot 用来记录当时购买商品所对应的地址, 在之后发生变化不会影响之前的订单。

表 3-5 订单表

字段名	中文名	类型	键/约束	说明
id	订单编号	String	主键	订单唯一标识
orderNo	订单号	String	唯一	业务订单编号
userId	用户编号	String	外键→User.id	下单用户
totalAmount	订单总额	Decimal(10,2)	—	总支付金额
paymentMethod	支付方式	String	—	支付渠道（支付宝、微信等）
paymentStatus	支付状态	String	—	如待支付、已支付
status	订单状态	OrderStatus	—	订单生命周期状态
note	订单备注	Text	—	用户备注信息
addressSnapshot	地址快照	Json	—	下单时地址快照
createdAt	创建时间	DateTime	—	数据创建时间

updatedAt	更新时间	DateTime	—	数据更新时间
-----------	------	----------	---	--------

（6）订单详情表（order_items）

用于存放订单详情，主键是 id，用 order_id 连接 orders 表，用 book_id 连接 books 表。有 title_snapshot、price_snapshot、quantity、subtotal 等字段，用来存储商品快照，保证订单有历史的一致性和可追溯性。

表 3-6 订单项表

字段名	中文名	类型	键/约束	说明
id	订单项编号	String	主键	订单明细唯一标识
orderId	订单编号	String	外键→Order.id	所属订单
bookId	图书编号	String	外键→Book.id	关联图书
titleSnapshot	书名快照	String	—	下单时图书标题
priceSnapshot	单价快照	Decimal(10,2)	—	下单时图书单价
quantity	数量	Int	—	购买数量
subtotal	小计金额	Decimal(10,2)	—	明细小计

3.4 系统安全设计

为了保证用户数据安全以及系统运行稳定性，本系统在身份认证、密码存储、权限控制、恶意脚本防护以及请求安全等方面进行了设计，并结合 JWT、bcrypt、CORS、CSRF 等技术实现基础安全保障机制，以减少常见 Web 安全问题对系统造成的影响。

3.4.1 JWT 身份认证机制

本系统使用基于 JWT 无状态的身份认证来对用户的登录进行处理，在用户完成注册或者登录之后，由服务端利用 jose 组件生成 JWT 并用 HS256 算法生成一个身份标识符，该标识符中含有用户的 id、role、email、name 等一些基本信息并且设置了 7d 过期时间以保持用户会话。

用户后续访问系统时，服务端会统一从 Cookie 中获取 JWT 进行验证，解出当前登录用户的用户信息，然后用该用户信息对请求进行鉴权，达到统一的身份认证的目的。

为保障用户账号安全，系统不对明文密码进行保存，而对用户的凭证进行加密后保存。

3.4.2 密码加密存储机制

在用户注册过程中，系统使用 bcrypt (bcryptjs) 对密码进行加盐哈希处理并且设置成本因子为 10，然后将其存储到数据库中；在用户登录时，系统调用 bcrypt.compare 方

法来比较输入的密码与存储在数据库中的哈希值是否相同，只有当两者一致时才允许用户进入系统。使用这种方法之后，即使数据库被泄露，攻击者也无法得到用户的明文口令，所以可以有效地防止大批量地猜解账户密码，从而保证系统的最基本的身份鉴别安全性。

3.4.3 权限控制机制

对于权限管理上，系统采取"身份认证+角色授权"的方式，针对不同的用户设置其可访问范围来防止出现越权的问题。

一般的业务接口在调用之前，都会使用 `getCurrentUserOrThrow()` 检查当前用户是否已经完成登录验证，如果未登录则不允许访问。一些涉及到后台管理的操作，还会用到 `getCurrentAdminOrThrow()` 对管理员身份进行检查，若不具备相应权限，则会返回 403，拒绝非法访问。

除了对接口权限进行检测之外，在后端页面入口处还存在一个服务端前置拦截，在普通用户试图访问后台页面时，会被跳转到后台登录页面从而防止非授权访问。

从数据的角度上，系统对于一些接口也做了相应的数据隔离处理，在订单详情查询中，一般用户只能看到自己的订单信息，而管理员可以查看所有订单的信息，通过对页面、接口及数据的不同维度进行管控，实现系统的权限控制。

3.4.4 XSS 风险防护机制

对于 XSS（Cross Site Scripting）的风险，系统在前后端均做相应的防护措施来降低被恶意脚本所造成的危害。

前端部分使用 React 默认渲染方式，系统会将页面中插入的内容进行转义防止用户输入的内容被当作代码解析执行，在后端也会对接收到的部分用户的输入做一些简单的清理工作，比如移除 HTML 标签等来降低恶意脚本存储在数据库中的可能性。

同时，认证令牌统一保存在 HttpOnly Cookie 中，而不是使用 `document.cookie` 暴露给前端脚本读取，所以在某些 XSS 情况下可以减少用户的账户被窃取的风险。

除此之外，系统还启用了 CSP（内容安全策略），通过对脚本、资源及链接来源进行约束来降低非法脚本被调用的机会进而保证网页正常运行。

3.4.5 安全请求头与 CORS 控制

为提高系统的安全性，在整个项目的全局中间件中统一设置了 CORS 以及安全响应头来管理跨域请求以及浏览器的安全问题。

对于跨域访问，系统支持由源通过 APP_ORIGIN、APP_URL 或者本次请求来源决定，并且打开 Allow-Credentials 使浏览器可以发送带凭据的请求。另外，对于 Content-Type、Authorization、X-CSRF-Token 这些请求头还有 GET、POST、PUT、PATCH、DELETE、OPTIONS 这些请求方式都进行了明确说明以便保证整个请求是完整并且可控。

对于浏览器的安全设置，在浏览器中设置了 X-Frame-Options: DENY、X-Content-Type-Options: nosniff、Referrer-Policy 以及 Permissions-Policy 的安全响应头来避免点击劫持、资源嗅探或隐私泄露等风险。

同时，系统通过设置 CSP 策略，默认 src、script-src、connect-src、img-src、font-src 以及 frame-ancestors 等资源的可访问范围从而减少被加载恶意资源的风险；对于 OPTIONS 预检请求，系统也有一致的操作以便跨域请求可以顺利进行。

除此之外，系统还实现基于 Cookie 和请求头一致性校验的 CSRF 防护措施（双提交 Cookie）。对于有新增、编辑或者删除的操作的接口，在发起请求前都需要做 CSRF 校验，只有校验成功后才能继续发起请求，保证接口的安全性。

第 4 章 系统实现与测试

4.1 开发环境搭建

为了保证系统开发、调试以及运行时的一致性，在开发过程中对运行环境进行了统一设置。系统的开发是基于 macOS 开发，在开发中使用的开发工具有 VSCode 和 Node.js，通过它们来实现项目的开发、测试等。由于 Next.js 14 对运行环境有要求，因此推荐使用 Node.js 18 或更高版本，以便更好地发挥框架的功能。

在依赖管理上使用的是 pnpm@10.12.4，相比于传统的 npm，在依赖安装速度及磁盘空间占用上有一定优势，在依赖上主要是 Next.js 14.2.15、React 18.3.1、Redux Toolkit 2.2.8 等前端相关库以及 Prisma 5.19.1 作为 ORM 并与 MySQL 数据库通过 DATABASE_URL 进行连接。

项目开始之前，要进行前期准备，首先要用命令 pnpm install 安装项目所需要的包，然后把 .env.example 文件拷贝并重命名为 .env，设置 DATABASE_URL、JWT_SECRET、APP_URL 以及 APP_ORIGIN 等环境变量，在数据库中用命令 pnpm db:push 创建表结构，再用命令 pnpm db:seed 插入基础的数据，在这之后就可以用命令 pnpm dev 启动开发版，在浏览器打开 <http://localhost:3000> 查看效果。

4.2 系统测试

为了验证系统功能的正确性、稳定性与安全性，本系统从功能测试、接口测试以及安全测试三个方面进行了综合测试。

4.2.1 功能测试

功能测试是对系统的各个部分是否符合要求进行检查。包括用户注册、登录、浏览图书、查找图书、添加到购物车、下单等功能。

在测试中，以不同用户的操作路径来对系统的各项功能逐一检查，发现所有功能均可正确响应用户的指令，显示的数据也都是正确的，状态的变化也是即时的，不存在明显的功能问题或者逻辑错误等，整个系统的功能满足需求。

表 4-1 功能测试

用例编号	测试内容	操作步骤	预期结果	测试结论
U001	正常注册登录	填写合法信息注册，使用账号密码登录	注册入库，生成 JWT，成功跳转首页	通过
U002	分类筛选图书	首页选择指定图书分类	只展示对应分类上架图书	通过
U003	关键词模糊搜索	输入作者关键字检索	匹配相关图书并分页展示	通过
U004	添加商品	图书详情点击加入购物车	购物车数量增加，数据库生成记录	通过
U005	修改商品数量	购物车修改商品购买数量	价格实时重新计算，数据同步后端	通过
U006	提交订单模拟支付	勾选商品、选择地址提交订单	生成订单、扣减库存、订单标记已支付	通过
U007	新增收货地址	个人地址页新增收货信息	地址成功保存，可设为默认地址	通过
A001	管理员账号登入后台	输入管理员账号密码登录	校验角色，成功进入管理首页	通过
A002	新增上架图书	后台填写图书信息并上架	数据入库，前端页面可查看该书	通过
A003	后台变更订单状态	管理员将已支付订单改为已发货	订单状态前后端同步更新	通过

4.2.2 接口测试

接口测试是对系统的后端 API 是否稳定可靠进行检查。本项目使用 Next.js Route Handlers 搭建 RESTful 风格的接口，在 Postman 以及编写自动化请求脚本对这些接口进行测试。

测试内容有用户认证接口、图书查询接口、购物车接口以及订单接口等。主要测试各个接口对不同的请求方式（GET、POST、PUT、DELETE）返回的结果是否正确，还有参数校验、错误处理以及返回的状态码等是否符合要求。

测试结果显示，系统的接口工作正常，可以正常地响应合法请求，对于非法参数或者错误请求也会给出合理的提示信息，具有一定的鲁棒性。

表 4-2 接口测试

用例编号	接口地址	请求方式	测试入参	预期返回结果	测试结论
I01	/api/auth/register	POST	name、email、pwd (6 位以上)	success:true, 返回用户信息，写入登录	通过

				Cookie	
I02	/api/auth/login	POST	email、pwd	success:true, 返回用户信息 + 购物车数量	通过
I03	/api/books	GET	keyword = 历史、page=1	success:true, 返回图书列表 + 分页数据	通过
I04	/api/cart	POST	bookId、quantity=1	success:true, 返回购物车总数量	通过
I05	/api/cart/{itemId}	PATCH	quantity=3	success:true, 返回最新购物车列表	通过
I06	/api/addresses	POST	receiver、phone、省市区、详细地址	success:true, 返回新增地址数据	通过
I07	/api/orders	POST	addressId	success:true, 返回订单编号与订单 ID	通过
I08	/api/upload/avatar	POST	form-data 携带图片文件	success:true, 返回头像 URL 地址	通过
I09	/api/admin/books	POST	书名、作者、分类、价格、库存等	success:true, 新增图书入库	通过
I10	/api/admin/orders/{id}	PATCH	status=SHIPPED	success:true, 订单状态更新成功	通过

4.2.3 安全测试

为了保证系统的数据安全及用户的隐私，在本系统中对常见的 Web 安全问题进行了一次专门的安全测试，包括 JWT 无权限访问测试、非法请求测试以及 XSS 注入测试等。

（1）JWT 未授权访问测试

用未登录状态下向需要认证接口发起请求来检查系统能否正确阻止这些请求。测试发现系统可以成功检测到缺少或无效 token，也会响应一个 401 未授权的状态码，以阻止未授权用户获取敏感信息。

（2）非法请求测试

利用构造非法参数请求接口，比如错误 ID、缺少字段或者数据类型不符等，检测系统健壮性，在此过程中发现系统可以正常阻止非法请求并给出一致错误信息，不会导致系统挂起或者错误数据写入数据库中。

（3）XSS 输入测试

在输入框中注入包含脚本代码字符串（例如 `<script>alert(1)</script>`），检查该网站是否具有跨站脚本漏洞，在前后端都对接收到的数据进行过滤以及转义之后，发现该脚本不能被运行，从而避免 XSS 攻击。如图 4-1 所示。



图 4-1 XSS 攻击示例

4.2.4 测试结果分析

对系统进行全方位测试后可得如下结果：从功能上来看系统实现了设计要求，在各个部分工作正常并且操作流程合理；从接口上看具有良好的规范性和异常处理能力，可以很好地处理各种请求；从安全角度看使用 JWT 认证以及对输入进行过滤等方式防止了非法访问以及常见的 Web 攻击。

总体来说，本系统从功能完整度、系统稳定性和安全性等都满足设计要求并且有良好的实用性和扩展性。

第 5 章 总结与展望

本文主要针对“基于 Next.js 全栈网上书店管理系统”进行设计与研究，在当前电商发展形势下及图书销售行业对信息化的需求背景下，对网上书店系统实现过程进行探讨与实现。课题包括系统的需求分析、各个功能的设计、数据库设计、前后端开发及系统测试等部分，最后完成一个可以正常使用的基本的全栈网上书店管理系统。

在系统设计之前，我们先了解网上书店的工作过程并结合用户的实际需求，把系统划分为不同的用户角色及其所需要的功能。系统主要有两个角色：普通用户和管理员。普通用户可以进行注册登录、浏览商品、按类查询、通过关键词查找书籍、查看书籍信息、添加到购物车、下单购买和查看订单情况等；而管理员可以对书籍进行管理、对订单进行处理、对类别进行增删改查以及对用户的信息进行维护等。根据不同的权限分配，可以使系统的功能满足网上书店的基本需求。

在系统实现过程中，本文基于 Next.js 开发工具，采取前后端一体方式进行开发，前端使用 React 组件化思想进行页面编写，有利于提高代码重用率并且便于后期功能修改与升级；而后端根据各个接口功能实现用户注册登录、图书管理、购物车管理和订单管理等功能，让整个系统具备相应基本功能。从数据库角度来说，选用 MySQL 存储系统内各种信息并结合具体业务情况制定相关表结构，来连接用户、图书、订单及购物车等内容，从而保证系统中数据的安全可靠^[15]。

为了检验该系统可行性和可靠性，本文也对该系统进行相关功能测试和压力测试，在功能测试中主要针对用户登录、图书查询、加入购物车、提交订单及后台图书管理等功能进行测试，结果表明各个功能均可正常使用并且可以实现整个网上购物的基本流程；而在压力测试中则采用一定数量用户同时浏览网站的方式测试系统性能，结果显示该系统对于小到中等规模的并发量有良好的表现力，页面加载速度快符合一般的网络书店的要求。

从最终实现效果来看，本系统主要具有以下几个特点：

（1）采用全栈开发模式，提高开发效率。

系统基于 Next.js 进行统一开发，在相比于传统的前后端分离方式上简化了接口联调以及项目的后期维护的工作量从而提升开发效率^[16]。

（2）系统功能较为完整。

系统基本上实现了一个网上书店的基本功能，如浏览书目、查找书籍、购物车、订

单管理等，并且管理员可以对所有的书籍以及订单进行统一管理，具有一定的实用性。

（3）采用模块化设计思想。

开发过程中，把用户、图书、订单、购物车等业务进行拆分到不同的模块中，每个模块的功能明确，在后续维护及功能增强上更易实现。

（4）数据库结构设计较合理。

系统根据实际情况进行数据库的设计，在一定程度上避免了数据冗余的问题并且提高了对数据的检索速度以及管理能力。

虽然目前系统已达到预期目的并且实现网上书店大部分主要功能，但是由于开发时间短和个人技术水平等原因，系统还有许多不足之处需要进一步改进。

第一，在支付上，当前的系统是模拟支付，未实际连接到支付宝、微信支付等第三方支付平台，所以不能构成一个完整的交易过程，在以后可加入第三方支付接口使系统的支付更加真实有效。

对于图书推荐这一模块而言，当前系统主要是以分类浏览以及关键词搜索来使读者找到相应的书籍，个性化推荐尚有改进余地，在以后的工作中可以根据读者的浏览行为、购物车信息以及收藏夹进行分析，采用协同过滤或者智能推荐等方法给读者提供更契合个人爱好的图书推荐，提高用户体验。

从系统的性能来看，当前系统适合小到中等规模的访问量，在访问人数逐渐增多的情况下，系统的响应速度就会受到数据库性能及服务器资源的影响，在接下来可以采用数据库索引优化、使用 Redis 缓存、引入消息队列以及读写分离等方法提升系统的性能，在大量并发的情况下也能保证良好的体验并且使整个系统的运行更加流畅。

从系统安全的角度来看，在现有的基础上已经实现了简单的身份验证以及权限管理等功能，但是安全性还有很大的提升空间，后续可增加对接口请求合法性检查，防范 SQL 注入、XSS 等恶意操作来进一步保障系统的正常运行。

总体而言，本课题实现了基于 Next.js 全栈网上书店管理系统开发，基本实现网上书店主要功能，在开发过程中对于现代 Web 开发技术、数据库设计及系统测试有更深认识，同时也提高自身全栈开发水平，获得一定软件系统分析与项目开发经验。

将来，随着互联网技术及电子商务的发展，网上书店系统仍有很大的改进之处，在以后的工作中可进一步利用云计算、大数据分析、人工智能推荐等方式改善系统的结构与体验，使其更加智能化、个性化以及高效，提高其实用性。

参考文献

- [1] 王华威. 基于 Node.js 的网上书店设计与实现[J]. 中国新技术新产品, 2020(22):43-46. DOI:10.13612/j.cnki.cntp.2020.22.015.
- [2] 于虹, 甄彤, 祝玉华. 浅析网上书店的实现[J]. 福建电脑, 2019, 35(7):71-73. DOI: 10.16707/j.cnki.fjpc.2019.07.024.
- [3] 秦佳. 基于 MVC 模型的网上书店系统设计与实现[J]. 电子技术与软件工程, 2019(5):44. DOI: 10.20109/j.cnki.ets.2019.05.036.
- [4] 苗省, 黎瑞源. 基于 FastAPI 与 React 的核心种质分析系统设计与实现[J]. 电脑知识与技术, 2025, 21(31):50-53. DOI:10.14004/j.cnki.ckt.2025.1577.
- [5] 张新海, 蔡会霞. 基于 React 的高校信息化项目管理系统的前端设计与实现[J]. 信息技术与信息化, 2025(7):53-56.
- [6] 钟佳伶. 基于混合式教学模式的前端框架应用开发课程教学改革与实践研究[J]. 电脑知识与技术, 2024, 20(13):170-173. DOI:10.14004/j.cnki.ckt.2024.0642.
- [7] 王金柱. TypeScript+React Web 应用开发实战[M]. 北京: 电子工业出版社, 2024:507.
- [8] 凌杰. Node.js 后端全程实战[M]. 北京: 人民邮电出版社, 2023:371.
- [9] 巴音查汗, 廖建尚, 张凯. Web 前端应用开发技术[M]. 北京: 电子工业出版社, 2022: 289.
- [10] 韩岗, 王伶俐, 李晋华. React 基础教程[M]. 北京: 人民邮电出版社, 2022:222.
- [11] 吴宇鹏. Web 应用前端框架的设计与应用[J]. 信息与电脑(理论版), 2021, 33(18):128-130.
- [12] Lazuardy M F S, Anggraini D. Modern front end web architectures with react.js and next.js[J]. Research Journal of Advanced Engineering and Science, 2022, 7(1):132-141.
- [13] Webify IT Solutions Enhances Offerings with Full Stack Web Development Services for Business Growth[J]. M2 Presswire, 2026.
- [14] Rodriguez A. RESTful API Design Principles: A practical guide to the architecture of the web (English Edition)[M]. BPB Publishers, 2025. DOI:10.0000/9789365891409.
- [15] Philip A. Full Stack Web Development: Mastering Web Development from Client to Server-Side Technologies[M]. Packt Publishing Limited, 2024. DOI:10.0000/9781836644927.
- [16] Parningotan L M. Design Pattern Evaluation on A RESTful API Wrapper: A Case Study of Software Integration with An Internet Payment Gateway using Model-Driven Architecture[J]. Journal of Information Technology and Computer Science, 2019, 4(3):222. DOI:10.25126/jitecs.201943107.

致 谢

光阴似箭，本人所作毕业论文《基于 Next.js 的全栈网上书店管理系统》的研究与撰写工作也即将完成，在整个课题研究、系统实现及论文写作期间，我获得了很多老师的指导以及同学们的帮助，在这里我要对所有帮助我的人表示衷心的感谢。

首先，感谢我的导师吕国华老师，周策老师，在论文选题、研究思路、系统设计及论文写作等方面给予我悉心指导。从课题整体安排到论文撰写，从系统功能设计到技术细节处理，老师都提出很多中肯意见与建议，使我能较好完成毕业设计。导师一丝不苟治学态度、任劳任怨工作态度和渊博知识，对我影响非常大，对我的以后学习和工作起到了良好示范作用。

在毕业论文即将完成之时，我诚挚地向指导老师表示感谢，在课题的研究以及论文写作的过程中，老师一直给予我悉心的教诲。从题目选定，系统的构思到论文的修订，老师都给我很多中肯的意见，使这个课题得以顺利完成并且也使我在专业知识以及研究水平上都有很大提高。

同时，感谢学院各位专业教师在大学期间的悉心培养。在课程学习过程中，老师们系统讲授了程序设计、数据库原理、软件工程以及 Web 开发等相关知识，为本课题的开展提供了扎实的理论基础。特别是在毕业设计开发过程中，课堂所学内容得到了进一步实践，也使自身分析问题和解决问题的能力得到了一定提高。

在毕业设计进行过程中，有一部分同学参与系统联调以及功能测试，这件事情给我留下深刻印象。许多问题是不会出现在“写代码”这个环节上出现，在反复测试接口、调试购物车逻辑过程中才会发现。因此我们在交流过程中不断完善各个方面的实现方式，有的甚至拆分重写。这说明一个系统的正常运作并不是某一个人完成工作就可以做到，还需要大家相互配合共同完善它。

家人的支持是在论文后期我才认识到它的价值。写作遇到瓶颈时期，一天的时间被分割成很多小块，注意力很容易被分散，在这样的情况下，家中给予我一个比较安静的环境，使我可以把更多的精力放在梳理以及论文撰写上。因为这个“无打扰”的环境本身就可以节省一部分无形的成本。

对于这篇论文来说，由于开发时间较短以及自身经验不足，在一些地方设计还很粗糙，特别是在异常处理以及一些特殊情况考虑上还不够完善，在多次自我测试中也发现这些问题，但是因为时间有限无法对其进行改进。因此，以后若有时间和精力对这个项

目进行进一步完善的话，我会首先把这些不太完善的实现加以完善并且优化一些功能。

最后，在这篇论文从开始的需求收集到最后完成的过程中，也离不开老师、同学们、朋友们和家人们的帮助。并没有故意拔高结论的意思，只是陈述了一个事实：许多重要环节不是单方面决定完成的，而是在不断的交流与改进中一步步发展起来的。

附 录

附录 A 数据库表结构设计

附 A1.1 数据库概述

数据库类型：MySQL

ORM 工具：Prisma

逻辑模型来源：prisma/schema.prisma

主键策略：字符串主键（cuid）

时间字段策略：createdAt（创建时间）、updatedAt（更新时间）

附 A1.2 枚举类型设计

表 A1 用户角色枚举

枚举值	含义
USER	普通用户
ADMIN	管理员

表 A2 图书状态枚举

枚举值	含义
DRAFT	草稿
PUBLISHED	已发布
ARCHIVED	已归档

表 A3 订单状态枚举

枚举值	含义
PENDING	待处理
PAID	已支付
SHIPPED	已发货
COMPLETED	已完成
CANCELLED	已取消

附 A1.3 约束与索引设计

表 A4 主键约束

表名	主键字段
User	id
Address	id
Book	id
CartItem	id
Order	id
OrderItem	id

表 A5 唯一索引

表名	唯一字段/组合
User	email

Book	slug
Order	orderNo
CartItem	(userId, bookId)

表 A6 外键约束

子表	外键字段	父表字段	删除策略
Address	userId	User.id	Cascade
CartItem	userId	User.id	Cascade
CartItem	bookId	Book.id	Cascade
Order	userId	User.id	Cascade
OrderItem	orderId	Order.id	Cascade
OrderItem	bookId	Book.id	Restrict

附 A1.4 表间关系说明

- 1. User 与 Address：一对多，一个用户可维护多个收货地址。
- 2. User 与 CartItem：一对多，一个用户可对应多条购物车记录。
- 3. Book 与 CartItem：一对多，一本图书可出现在多个用户购物车中。
- 4. User 与 Order：一对多，一个用户可生成多个订单。
- 5. Order 与 OrderItem：一对多，一个订单包含多个订单项。
- 6. Book 与 OrderItem：一对多，一本图书可关联多个订单项记录。